

Application de l'intelligence artificielle au domaine de l'expérience utilisateur : comment utiliser les technologies de machine learning pour interpréter des résultats de questionnaires Attrakdiff ?

Carolina Courtin

Projet de bachelor en systèmes d'information et science des services

Centre Universitaire d'Informatique, Université de Genève

Projet supervisé par le Pr. Patrick Roth

30 juin 2024



Résumé

L'expérience utilisateur (ou user experience, UX) est aujourd'hui une dimension importante dans la réalisation de services puisqu'elle permet de créer des connexions durables avec les utilisateurs. Ce domaine donne lieu au développement d'interfaces de tout types comme celles d'un site web ou d'un panneau numérique sur une imprimante. Plusieurs techniques existent pour évaluer l'efficacité de ces interfaces et souvent elles aboutissent à des résultats quantitatifs qu'il faut encore interpréter en détail pour comprendre les changements précis à apporter dans le système. Cette étude vise à trouver un moyen de faciliter cette interprétation en utilisant des technologies de machine learning. Nous en utiliserons plusieurs et tenteront d'évaluer laquelle est la plus apte à réaliser cet objectif.

Table des matières

1	Introduction	3
2	Etat de l'art	4
2.1	Evaluation d'interface en expérience utilisateur	4
2.2	Interprétation des résultats d'évaluation	7
2.3	Machine learning appliqué au domaine de l'expérience utilisateur	8
2.4	Question de recherche	8
3	Proposition conceptuelle	9
3.1	Champ d'application	9
3.2	Démarche	10
3.2.1	Préparation des données	10
3.2.2	Choix et entraînement du modèle de machine learning	10
4	Solution développée	16
4.1	Outils utilisés	16
4.2	Strcuture du projet	16
4.3	Traitement des données	17
4.4	Entraînement des modèles	19
4.5	Résultats et choix du modèle	20
4.6	Application web	22
5	Evaluation	25
6	Conclusion	26
7	Annexe	27

1 Introduction

L'expérience utilisateur désigne la qualité de l'interaction entre un utilisateur et un système via une interface. C'est un critère qui doit attentivement être travaillé lors du développement d'un produit puisqu'il va avoir un impact déterminant sur celui-ci, comme notamment attirer un maximum d'utilisateurs [1]. Par interface, on comprend tout artefact qui se trouve entre l'humain et la machine. Il existe un grand nombre d'interfaces qui peuvent illustrer cette définition comme par exemple la page d'un site web, le panneau numérique d'une imprimante ou encore la manette d'une console de jeux vidéos. Les interfaces sont très variées mais dans le cadre de ce projet, nous avons choisis de nous concentrer sur les interfaces de type sites et applications web.

Le développement de ces interfaces se fait en plusieurs parties : il y a d'abord une étape d'"empathie" où le développeur va se mettre dans la peau de l'utilisateur pour comprendre ses besoins lorsqu'il interagira avec le système. Ensuite il faut définir ces besoins et les mettre en oeuvre dans un prototype puis ce prototype est testé, étape cruciale du développement d'interfaces. En effet, c'est l'étape qui va permettre de comprendre en détail les éléments qui doivent être modifiés afin d'avoir un meilleur rendu.

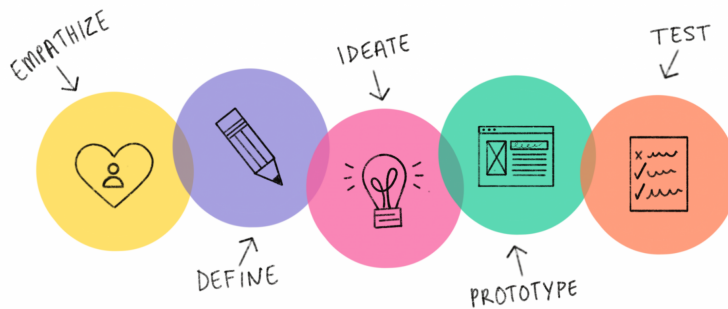


FIGURE 1 – Etapes de développement d'une interface

Les tests se penchent sur l'évaluation de plusieurs aspects du système, notamment les aspects ergonomique et esthétique. Les techniques d'évaluation sont diverses : la réalisation d'interviews qui permet de directement récolter l'avis des utilisateurs sur le système, la réalisation de tâches par les usagers sur l'interface pour évaluer à quel point celle-ci est utilisable (en utilisant aussi des métriques comme le temps de réalisation des tâches) et l'utilisation de questionnaires. Chacune de ces techniques va permettre de mener une évaluation plus ou moins détaillée dépendant du type d'interface que l'on va tester. Dans le cadre de cette étude, nous nous pencherons sur l'utilisation des questionnaires. Nous verrons au cours de l'étude que bien que les questionnaires sont une méthode efficace d'évaluation d'interfaces, leur utilisation requiert également de pouvoir traduire les résultats souvent numériques comme des moyennes, en éléments de l'interface à modifier. C'est pourquoi nous pensons que l'utilisation du machine learning peut être utile dans ce domaine : il pourrait permettre d'interpréter les données en sortie de questionnaires d'évaluation pour conclure sur les éléments spécifiques à retravailler sur l'interface (comme la fonte, les couleurs ou la disposition des éléments).

Dans un premier temps, nous ferons une revue de littérature qui s'appuie sur trois éléments importants de ce sujet : les différents types de questionnaires d'évaluation, l'interprétation des résultats de ces questionnaires et le machine learning appliqué au domaine de l'expérience utilisateur. Ensuite nous expliquerons de manière conceptuelle comment nous voulons répondre à la problématique dressée à partir de la revue de littérature. Nous décrirons ensuite dans une troisième partie la solution développée et les résultats pour enfin conclure sur les atouts du projet et les aspects à aborder dans le futur pour le renforcer.

2 Etat de l'art

Dans cette section, nous allons discuter des connaissances déjà établies dans trois domaines en lien avec ce projet : l'évaluation d'interfaces, l'interprétation de résultats d'évaluation et le machine learning appliqué au domaine de l'expérience utilisateur. L'idée est de comprendre ce qui a déjà été étudié dans ces trois domaines et les champs qui manquent de connaissances.

2.1 Evaluation d'interface en expérience utilisateur

Comme mentionné dans l'introduction, il existe plusieurs types d'évaluation d'interface, les plus connues étant les interviews, les réalisations de tâches sur le système et le remplissage de questionnaires. Dans cette étude, nous nous attarderons sur ce dernier type d'évaluation. Les questionnaires évaluent l'aspect physique de l'interface et non le champs technique. C'est-à-dire que l'on va tenter d'améliorer l'aspect visuel du site ou de la manette de jeux vidéo en laissant de côté sa technicité, champs qui sera évalué dans le cadre d'une autre discipline. Il existe plusieurs types de questionnaires. L'un des questionnaires les plus connu et le System Usability Scale (SUS) [2]. Ce questionnaire consiste pour l'utilisateur d'évaluer plusieurs énoncés concernant le système en leur donnant une note entre 1 et 5. A la fin, on calcule un score SUS entre 0 et 100 qui permet d'évaluer son utilisabilité. En plus de ce score, ce questionnaire permet aussi de calculer d'autres valeurs se basant sur les données récoltées au cours du test comme la moyenne, l'écart-type, etc.

System Usability Scale

	Pas du tout d'accord						Tout à fait d'accord
1. Je pense que je vais utiliser ce service fréquemment	☐	☐	☐	☐	☐		
	1	2	3	4	5		
2. Je trouve ce service inutilement complexe	☐	☐	☐	☐	☐		
	1	2	3	4	5		
3. Je pense que ce service est facile à utiliser	☐	☐	☐	☐	☐		
	1	2	3	4	5		
4. Je pense que j'aurai besoin de l'aide d'un technicien pour être capable d'utiliser ce service	☐	☐	☐	☐	☐		
	1	2	3	4	5		
5. J'ai trouvé que les différentes fonctions de ce service ont été bien intégrées	☐	☐	☐	☐	☐		
	1	2	3	4	5		
6. Je pense qu'il y a trop d'incohérence dans ce service.	☐	☐	☐	☐	☐		
	1	2	3	4	5		
7. J' imagine que la plupart des gens serait capable d'apprendre à utiliser ce services très rapidement.	☐	☐	☐	☐	☐		
	1	2	3	4	5		
8. J'ai trouvé ce service très lourd à utiliser.	☐	☐	☐	☐	☐		
	1	2	3	4	5		
9. Je me sentais très en confiance en utilisant ce service.	☐	☐	☐	☐	☐		
	1	2	3	4	5		
10. J'ai besoin d'apprendre beaucoup de choses avant de pouvoir utiliser ce service.	☐	☐	☐	☐	☐		
	1	2	3	4	5		

FIGURE 2 – System Usability Scale

Un autre questionnaire connu : le User Experience Questionnaire [3]. Au cours de ce test, le développeur sélectionne des affirmations s'inscrivant dans divers aspects à évaluer de l'interface (originalité, esthétisme, efficacité, etc.). Le testeur doit évaluer chacun de ces aspects et une note est ensuite calculée, comme pour le SUS, avec une formule mathématique particulière qui permettra de juger la qualité de l'interface.

Negative	1	2	3	4	5	6	7	Positive
obstructive								supportive
complicated								easy
inefficient								efficient
confusing								clear
boring								exciting
not interesting								interesting
conventional								inventive
usual								leading edge

<https://www.ueq-online.org/>

FIGURE 3 – User Experience Questionnaire

Il existe un troisième questionnaire assez répandu et détaillé : l'Attrakdiff [4]. Ce questionnaire évalue deux aspects principaux d'une interface : l'aspect pragmatique et esthétique. Pour ce faire, les participants doivent remplir une grille où ils notent entre -3 et 3 des paires d'adjectifs caractérisant l'interface. Par exemple pour la paire d'adjectif technique-humain, si un participant donne la note de 2.5 il considère que l'interface est plutôt humaine. Chaque paire de mot ou d'adjectif s'inscrit dans une catégorie d'évaluation : qualité pragmatique, qualité hédonique de stimulation, qualité hédonique d'identification et attractivité. Les notes sont ensuite utilisées pour calculer la moyenne de chaque catégorie et ces moyennes sont placées sur un diagramme pour mieux comprendre quels sont les aspects de l'interface qui doivent être retravaillés. Par exemple si l'aspect esthétique possède une moyenne trop basse, le développeur devra en déduire qu'il faut changer les couleurs ou la fonte sur le système. Les moyennes sont également utilisées pour construire un portfolio : le portfolio est un graphique qui se base sur la moyenne pragmatique et attractive et qui permet de situer l'interface quant à sa qualité. L'interface peut être par exemple trop orientée tâche ou trop orientée vers soi, l'idéal étant qu'elle se trouve dans la zone "désiré".

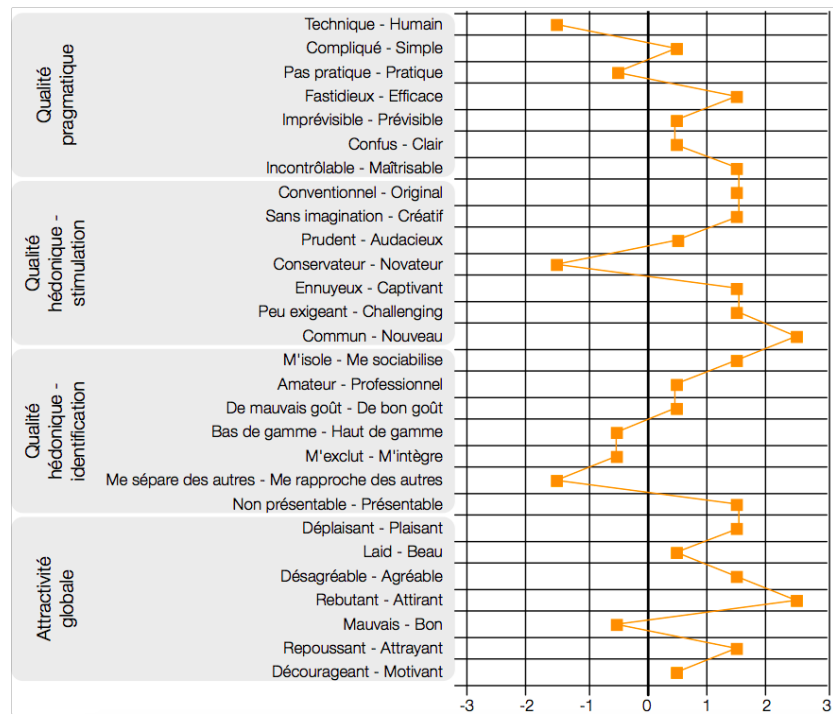


FIGURE 4 – Exemple de scores des paires d'adjectifs

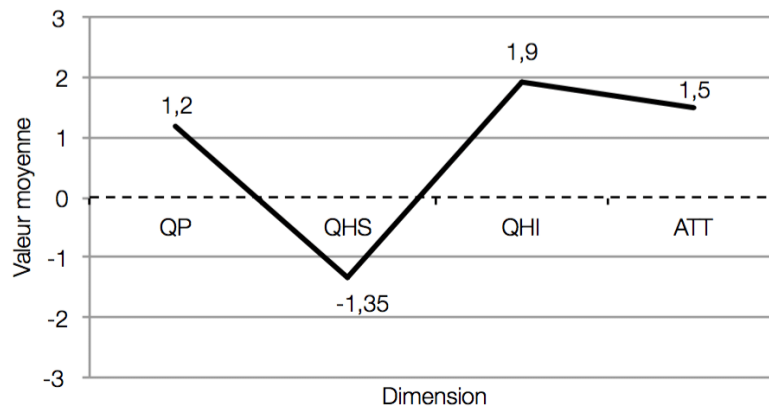


FIGURE 5 – Exemple de diagramme des valeurs moyennes

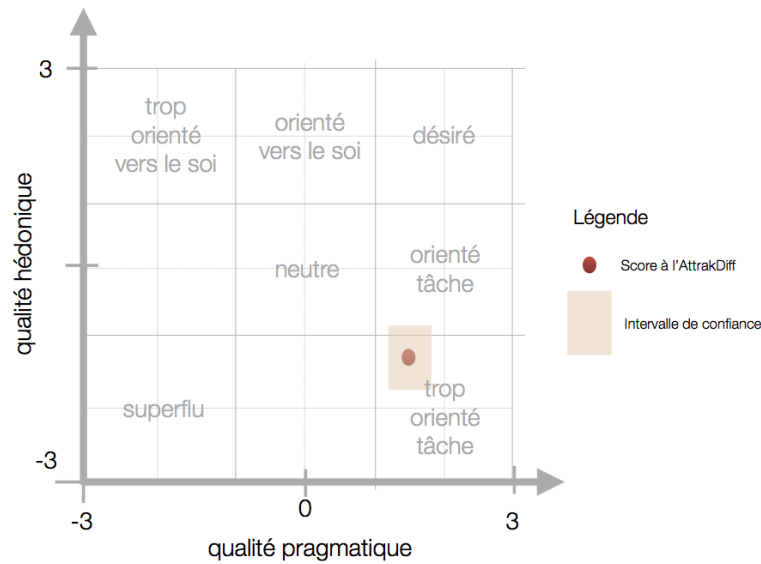


FIGURE 6 – Exemple de portfolio des résultats

L'Attrakdiff est le questionnaire qui permet d'avoir des résultats plus détaillés pour l'amélioration d'interfaces. Le fait d'évaluer des catégories précises dans le système permet ensuite de cibler sa modification en fonction des paradigmes relevés lors de l'évaluation. Bien que ce questionnaire donne également une valeur qualitative avec le portfolio (orienté-tâche, superflu, désiré...), les valeurs que l'on va surtout prendre en compte sont les valeurs numériques des moyennes et des scores attribués aux paires de mots. Ces valeurs devront être interprétées, c'est-à-dire qu'il faudra tenter de faire le lien entre une note reçue et l'élément concret à modifier dans l'interface.

2.2 Interprétation des résultats d'évaluation

Comme nous l'avons expliqué précédemment, les questionnaires d'évaluation d'interface aboutissent majoritairement sur des valeurs numériques qu'il faut ensuite traduire en éléments de l'interface à retravailler. L'interprétation de ces résultats doit se faire en se basant sur plusieurs points : les commentaires de chaque participant et l'analyse des résultats quantitatifs. Cette tâche doit en principe se faire par un humain puisqu'on doit analyser ces critères en détail tout en gardant à l'esprit le contexte dans lequel on se trouve (quelle est l'interface qui a été testée et quels sont les éléments importants). En se penchant sur la littérature de recherche existante, nous nous sommes rendu compte qu'il n'y a presque pas d'article de recherche traitant du sujet de l'interprétation de données quantitatives de questionnaires d'évaluation UX. L'une des seules études qui s'est penchée sur une manière d'interpréter ces résultats s'est basée sur l'interprétation d'un UEQ (User Experience Questionnaire) [5]. L'étude explique qu'il est possible de placer les caractéristiques évaluées dans un graphe, avec la performance comme axe des abscisses et l'importance de cette performance comme axe des ordonnées en utilisant les notes des testeurs. Le graphe est donc divisé en sections qui représentent l'état de la caractéristique (par exemple si l'aspect stimulation se trouve dans la zone du graphe "possible overkill", cela signifie que l'aspect est trop développé par rapport à d'autres, et donc que le service n'a pas besoin d'être trop technique par rapport à son but). Cette méthode est utile car elle permet de déterminer de manière plus précise le domaine qui doit être analysé. Cependant, il reste assez large et ne permet pas d'avoir un attribut de l'interface précis à modifier. En effet, nous cherchons à déterminer les éléments spécifiques de l'interface qui nuisent à l'expérience utilisateur, or cette technique ne permet pas d'atteindre un tel niveau de précision. C'est pourquoi l'utilisation de techniques machine learning pourrait être un moyen de pallier ce problème.

2.3 Machine learning appliqué au domaine de l'expérience utilisateur

Nous avons vu qu'une partie importante du travail de développement d'interface réside dans l'interprétation des résultats de tests. Contrairement à d'autres étapes, il est compliqué d'automatiser cette tâche puisqu'elle requiert d'avoir conscience du contexte dans lequel le service est développé. C'est pourquoi l'intégration du machine learning dans ce processus pourrait être intéressante.

Le machine learning est une technologie appartenant au domaine de l'intelligence artificielle. Son principe est qu'elle va apprendre à partir de données qu'on lui donne en entrée pour créer un schéma, schéma qu'elle réutilisera plus tard pour faire de la prédiction à partir de nouvelles données. Il existe plusieurs modèles de machine learning qui fonctionnent de manière différente et qui sont utilisés pour réaliser des tâches particulières. Cette technologie se répand dans de nombreux domaines et commence à se démocratiser en expérience utilisateur. Certains scientifiques encouragent même les professionnels du domaine à utiliser le machine learning [6].

Une étude a notamment démontré qu'il était possible de reconnaître les émotions des utilisateurs lors de la phase de test du service, en utilisant plusieurs algorithmes de machine learning (random forest, binary decision trees, AdaBoost et Multi Layered Perceptron) [7]. L'étude a démontré l'efficacité de cette technique, et permet de comprendre à quel point elle peut être profitable.

L'utilisation du machine learning, et plus précisément des réseaux Bayésiens, a également été utilisée pour modifier l'aspect d'un service en se basant sur le comportement de l'utilisateur pour en déduire ses préférences [8]. Même si l'étude s'est limitée à des modifications simples de l'interface (couleurs, contenu, etc.) cela montre encore le soutien aux développeurs que le machine learning peut apporter. Bien que le machine learning soit utilisé pour directement améliorer l'aspect d'une interface ou les sentiments des utilisateurs, rares sont les études qui se focalisent sur le traitement de questionnaires d'évaluation. L'étude la plus proche de ce sujet démontre la possibilité de prédire l'avis final des consommateurs sur un service en se basant sur leur résultats de questionnaire d'évaluation [9]. Pour ce faire, les chercheurs ont entraîné plusieurs modèles de machine learning à partir de deux datasets basés sur les réponses des testeurs (chaque dataset contenant 5 et 7 réponses de questionnaire respectivement). A l'issue de ce test, ils ont pu prédire la satisfaction des utilisateurs avec un taux de fiabilité de 0.90 et 0.93 pour chaque dataset avec le modèle SVM polynomial. Cette étude démontre qu'il serait possible d'appliquer un raisonnement semblable en utilisant des réponses de questionnaires pour prédire les points à modifier d'une interface.

2.4 Question de recherche

Les sections précédentes nous ont permis de comprendre ce qui a déjà été travaillé en recherche et ce qui n'a pas encore été traité. Nous avons vu que les technologies de machine learning commencent à être utilisées en expérience utilisateur pour effectuer différentes tâches. Cependant, nous avons souligné que le domaine de l'interprétation de résultats quantitatifs n'est quasiment pas traité. De plus, l'utilisation du machine learning concerne rarement l'application à des questionnaires. Aussi, nous avons souligné le fait que parmi les divers questionnaires d'évaluation existants, l'Attrakdiff semble être le plus précis. Le développement de cet état de l'art nous permet alors de dresser la problématique à laquelle ce projet va tenter de répondre : **Comment le machine learning peut-il aider à interpréter les résultats de questionnaires d'évaluation d'interfaces de type Attrakdiff ?**

Dans la section suivante, nous allons présenter une proposition conceptuelle pour répondre à cette problématique.

3 Proposition conceptuelle

La proposition que nous avons élaboré pour répondre à la problématique dressée dans la section précédente est la suivante : il faudrait choisir un modèle de machine learning assez performant qui s'entraînerait sur un ensemble de résultats d'évaluation quantitatifs et sur les conclusions trouvées par les développeurs à partir de ces résultats. Le modèle serait ensuite capable de prédire, à partir de nouvelles données numériques, les éléments de l'interface à modifier en se basant sur l'entraînement réalisé en amont. Plusieurs points importants doivent être pris en compte pour permettre la réalisation de ce projet. Nous les passerons en revue dans cette section.

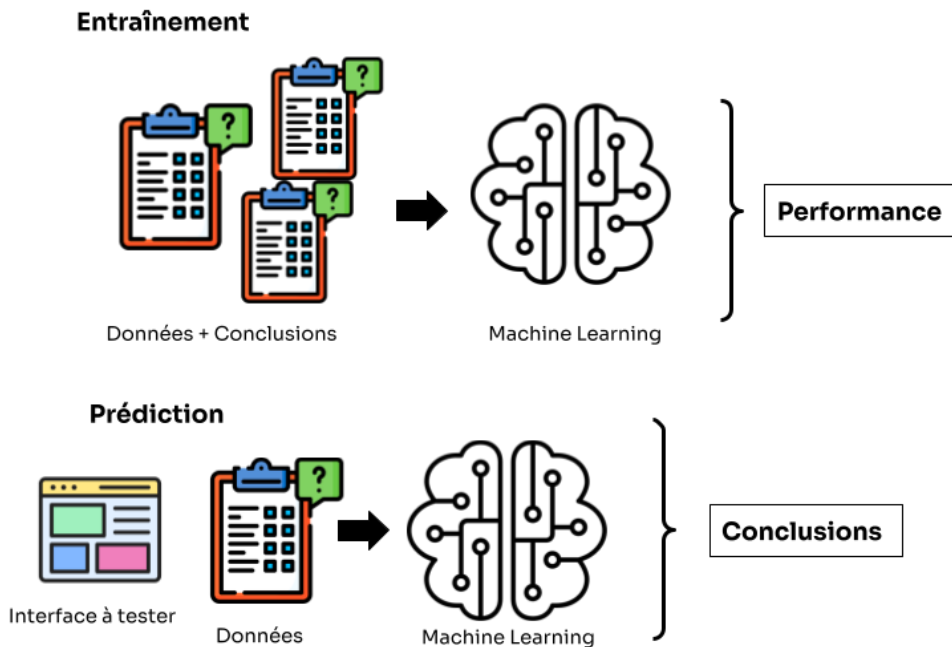


FIGURE 7 – Schéma de proposition

3.1 Champ d'application

Nous l'avons vu en introduction : il existe tout type d'interfaces. Aussi les tests d'interfaces ont généralement une démarche et des résultats attendus différents en fonction de la nature du système que l'on va tester. Cette différence se reflète dans le genre de test ou de questionnaire que l'on utilise lors de la phase d'évaluation. C'est pourquoi il a d'abord fallu choisir une classe de système sur laquelle se concentrer. Notre choix s'est porté sur les interfaces de type sites et applications web puisque ce sont des systèmes qui peuvent être plus ou moins riches en éléments ce qui permet d'avoir des résultats d'évaluation variés et intéressants. Ce sont également des objets d'étude où il est simple de retrouver des éléments particuliers du système comme des boutons, des icônes, des couleurs, etc.

Le développement de cette solution nécessite également d'utiliser des résultats de questionnaires d'évaluation d'interface de type quantitatifs. Tous les questionnaires présentés dans la section "état de l'art" peuvent donc fonctionner, cependant nous avons décidé de nous baser sur le questionnaire Attrakdiff. En effet ce questionnaire est celui qui possède le plus de valeurs et de précision dans son analyse.

Enfin, il a fallu choisir un modèle de machine learning avec lequel nous travaillerons pour interpréter les résultats d'Attrakdiff. Nous expliquerons en détail la démarche que nous avons suivie pour faire ce choix dans la section suivante.

3.2 Démarche

Nous allons maintenant expliquer en détail les différentes étapes à suivre pour réaliser la proposition présentée au début de cette section.

3.2.1 Préparation des données

Comme nous l'avons expliqué dans la section précédente, notre choix de questionnaire s'est porté sur l'Attrakdiff. Il a donc fallu dans un premier temps se procurer des résultats de questionnaires Attrakdiff sur l'évaluation de sites et d'applications web. Nous avons pu exploiter deux sources de données :

- * L'analyse de plusieurs articles scientifiques possédant une évaluation de type Attrakdiff [10] [11] [12] [13] [14]
- * Les résultats de travaux d'élèves du Pr. Patrick Roth dans le cadre du cours Interface Personne Machine où ces derniers ont eu comme devoir de créer une interface et de la tester avec le questionnaire Attrakdiff

L'idée est d'entraîner le modèle avec deux types de données différents : les 4 valeurs moyennes et les 28 scores de paires de mots des questionnaires Attrakdiff. Pour chaque questionnaire, le modèle doit aussi s'entraîner sur les modifications trouvées par le développeur à partir des données du questionnaire. Par exemple, l'une des données d'entraînement pourrait avoir une moyenne de qualité hédonique de stimulation très basse et comme conclusion relevée par le développeur, l'ajout de microinteractions sur l'interface.

En inspectant les sets de données, nous nous sommes rendus compte que beaucoup étaient incomplets. En effet il manquait souvent l'une des moyennes ou l'un des scores de paire de mots mais aussi les conclusions. En effet, si certaines évaluations possédaient toutes les valeurs quantitatives, il manquait souvent la conclusion du développeur, c'est-à-dire les éléments de l'interface qui doivent être changés selon lui en s'appuyant sur ces scores. Par exemple, si un questionnaire présentait une moyenne de qualité pragmatique très basse, nous nous attendions à avoir aussi une conclusion du type "modifier la disposition des boutons sur l'interface". Or pour avoir un résultat optimal, il est nécessaire d'avoir un maximum de données complètes. Par soucis de manque de données, il a donc parfois fallu interpréter les résultats quantitatifs nous même pour aboutir à des propositions de modification d'interface. Si la valeur ne pouvait pas être recalculée ou que notre interprétation était trop peu cohérente avec les résultats, le questionnaire correspondant était alors mis de côté. En plus du soucis de données incomplètes, nous nous sommes rendus compte d'un problème concernant les données des articles scientifiques : l'Attrakdiff était surtout utilisé pour tester l'aspect technique de l'interface. Or le but de notre projet est de rester sur l'aspect physique pour l'améliorer. Finalement, nous avons récupéré 12 résultats d'Attrakdiff avec les scores paires de mots complets et 15 résultats avec les valeurs moyennes complètes.

3.2.2 Choix et entraînement du modèle de machine learning

Il existe un grand nombre de modèles de machine learning. Les modèles sont utilisés pour réaliser diverses tâches comme le clustering ou la régression linéaire. Le but de notre projet est d'utiliser les données Attrakdiff pour aboutir à un problème de l'interface testée. C'est pourquoi nous avons décidé d'utiliser un modèle de classification. De plus, comme une interface peut avoir plusieurs problèmes, nous utiliserons un modèle de classification multiclasse, c'est-à-dire un modèle qui permet de retourner plusieurs classes pour une entité testée.

Dans la phase d'entraînement d'un modèle de classification, les données possèdent deux propriétés : les caractéristiques des données et les valeurs de sorties. Par exemple, si on entraîne un modèle à classer des pommes et des oranges, chaque fruit possède un ensemble de caractéristiques (le poids, la couleur, etc.) et une valeur de sortie qui correspond à la nature de la donnée (pomme ou orange). Dans notre cas, les caractéristiques correspondent aux valeurs numériques de l'Attrakdiff (moyennes ou scores pour chaque paire de mots) et les valeurs de sortie correspondent aux problèmes de l'interface trouvés (esthétisme, professionnalisme, etc.). Le modèle va d'abord s'entraîner sur ces données pour analyser les caractéristiques et les classes auxquelles les données caractérisées appartiennent. Ensuite, le modèle entraîné peut prédire, à partir des caractéristiques d'une nouvelle donnée, la classe à laquelle

elle appartient.

Nous avons testé 6 modèles de machine learning principaux afin de trouver le plus performant pour ce projet :

- * **Réseaux Bayésiens** : Ce modèle calcule les probabilités qu'une entité appartienne à une classe en se basant sur la dépendance de ses caractéristiques. Dans ce projet, nous avons utilisé un type de réseau bayésien particulier : le réseau Gaussien naïf. Ce type de réseau part du principe qu'il n'y a pas de dépendance entre les caractéristiques d'une donnée. Ces caractéristiques sont sous forme de valeur numériques à la différence du réseau bayésien classique où ces données peuvent prendre la valeur "oui" ou "non".

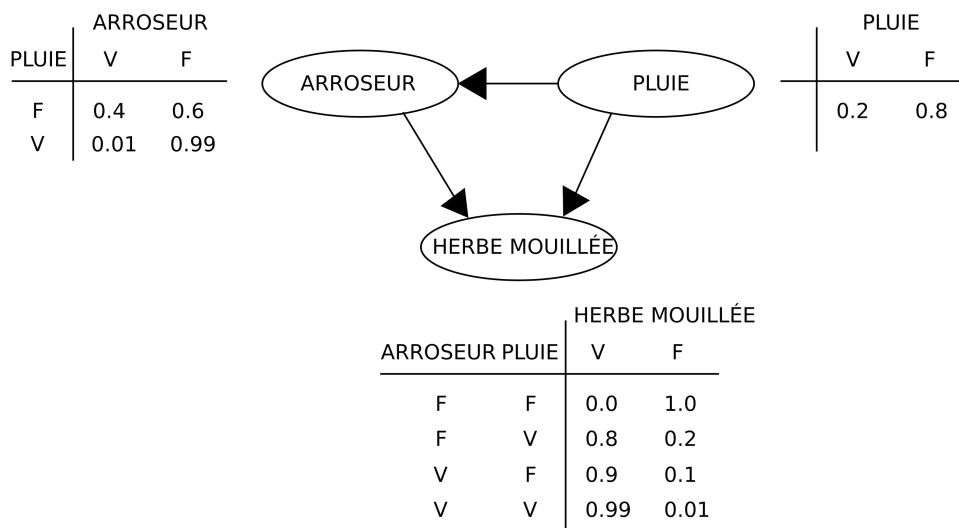


FIGURE 8 – Exemple de réseau bayésien

- * **Arbres de décision** : Ce modèle analyse chaque caractéristique d'une entité et se base sur ces données pour déduire la classe à laquelle elle appartient. C'est un modèle assez intuitif qui permet d'avoir une représentation graphique de la réflexion récursive du modèle.

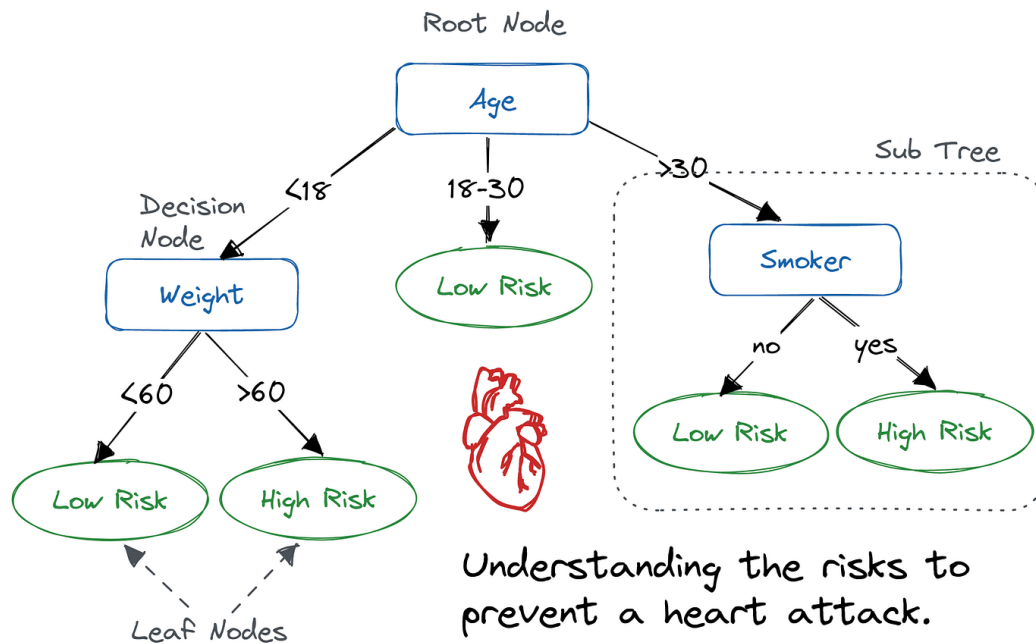


FIGURE 9 – Exemple d'arbre de décision

- * **Forêts Aléatoires (Random Forest)** : Les random forest entraînent plusieurs arbres de décision à partir de l'ensemble de données d'entraînement. Lorsqu'une prédiction doit être faite, les arbres se basent sur les caractéristiques de la donnée pour prédire la classe à laquelle cette donnée appartient. A la fin, tous les arbres votent pour décider de la classe finale à prédire.

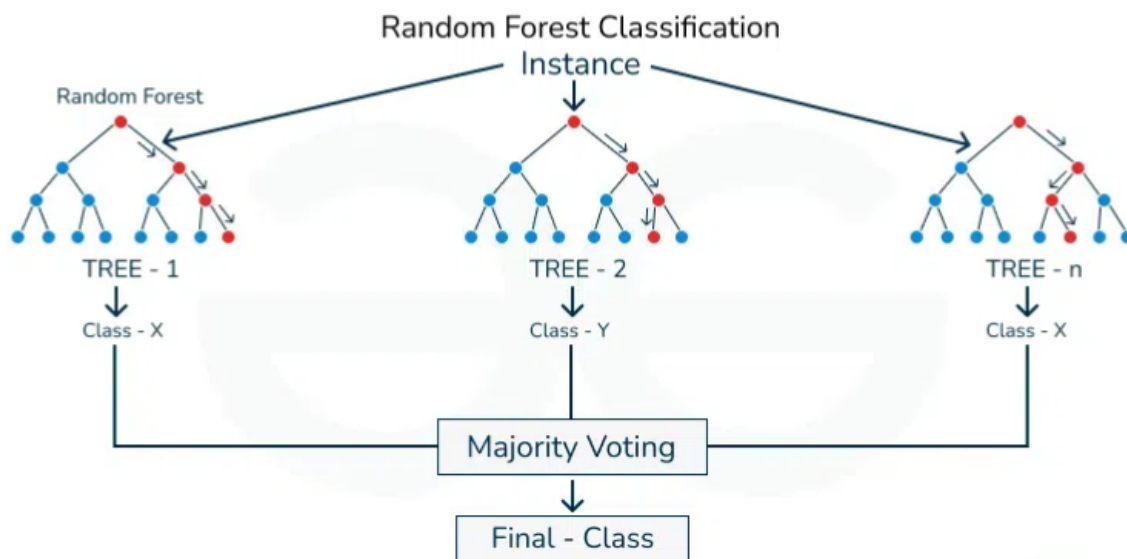


FIGURE 10 – Fonctionnement du Random Forest

- * **K-Nearest-Neighbors (KNN)** : Ce modèle utilise un plan à N-dimension, N correspondant au nombre de valeurs qui caractérisent une entité. Cette dernière est placée sur ce plan en fonction de ses caractéristiques. Le principe est que le modèle va calculer la distance entre cette entité et les K-valeurs les plus proches de lui, qui représentent en fait les K-valeurs ayant les caractéristiques les plus proches de la nouvelle entité. C'est ensuite la classe à laquelle appartiennent le plus de valeurs proches qui est retournée.

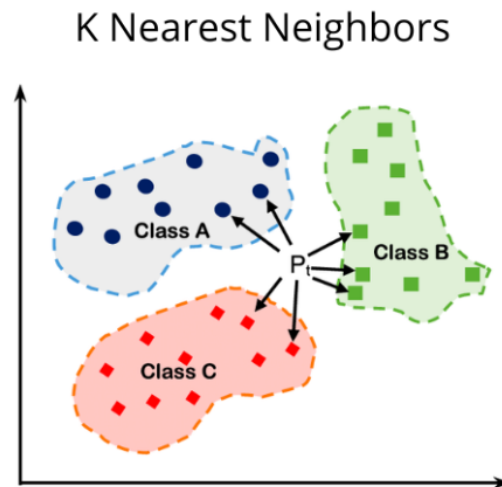


FIGURE 11 – K-Nearest-Neighbors

- * **Régression Logistique** : Ce modèle utilise une fonction sigmoïde pour convertir le point de donnée à prédire en probabilité. Cette fonction divise un plan en fonction du nombre de classe différentes et place les nouvelles données sur le graphe du côté de la classe calculée.

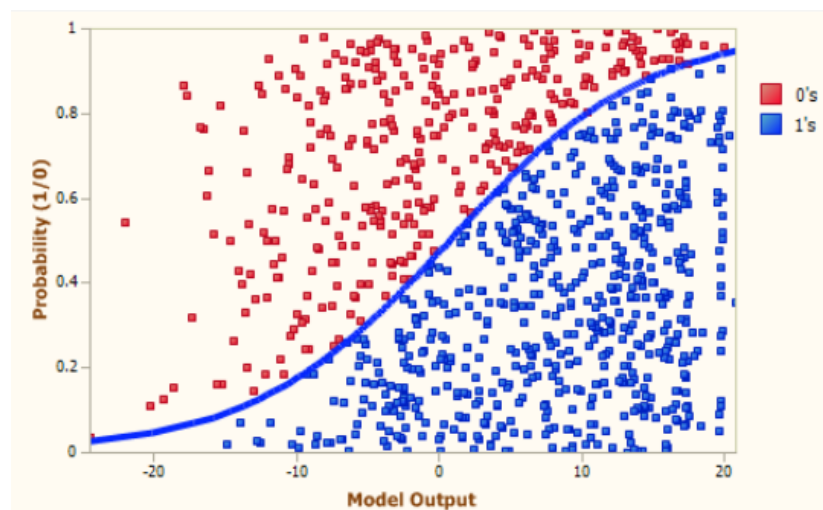


FIGURE 12 – Régression Logistique

- * **Machine à vecteur de support (SVM)** : Les machines à vecteur de support utilisent aussi un plan à N-dimension dépendant du nombre de caractéristiques des données. Lors de la phase d'entraînement, les données sont placées sur ce plan et l'algorithme va tenter de les séparer au maximum en fonction des classes auxquelles elles appartiennent. Un hyperplan est ensuite calculé pour délimiter les différentes classes de données sur le plan à N-dimension. Lorsqu'une prédiction doit être faite, le point de donnée est placé sur ce plan et la classe prédite dépendra de la position de la donnée par rapport à l'hyperplan.

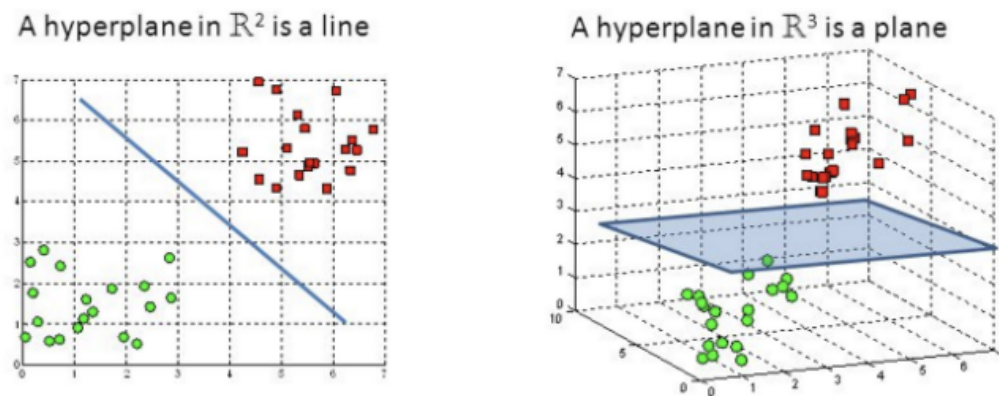


FIGURE 13 – Machine à vecteur de support

Il est important de noter que ces modèles ne sont pas spécifiquement créés pour la classification, on les utilise aussi pour réaliser d'autres tâches (régression, clustering...). Dans notre cas, il faut combiner la logique mathématique de ces modèles avec un autre algorithme pour permettre de réaliser un système de classification multiclasse. Nous avons testé deux de ces algorithmes :

- * **One-vs-Rest (OVR)** : Un modèle de classification complet est entraîné pour chaque classe. Ce modèle va à chaque fois prédire si l'entité fait partie de cette classe ou non. Chaque prédiction de classe est indépendante des autres.
- * **Classifier Chains (CC)** : Le principe est similaire à l'algorithme précédent mais dans ce cas, CC va aussi se baser sur les prédictions des classes passées.

Exemple : Si un modèle doit s'entraîner à distinguer des pommes et des oranges, les deux algorithmes vont créer deux classificateurs (un pour la classe "pomme" et un pour la classe "orange") et vont pour chaque classe analyser les attributs du fruit pour conclure s'il appartient à la classe ou non. Avec OVR chaque prédiction de classe est indépendante des autres mais avec CC, quand l'algorithme analysera l'appartenance du fruit à la classe "orange", il prendra aussi en compte l'analyse effectuée lors de la prédiction pour la classe "pomme". Cela permet de capturer des liens cachés entre les données et d'avoir, dépendant des données que l'on utilise, une meilleure prédiction.

Comme expliqué précédemment, ces algorithmes doivent être combinés à une logique mathématique d'un modèle de machine learning (par exemple l'algorithme Classifier Chains et le modèle de régression logistique). Nous avons également mentionné le fait que nous voulions tester le modèle sur les valeurs moyennes et sur les paires de mots de l'Attrakdiff. Au total, ce sont 24 modèles que nous avons testé en combinant les 2 algorithmes de classification multiclasse, les 6 modèles de machine learning et les 2 types de données Attrakdiff.

En machine learning, un modèle est entraîné sur une portion des données d'entraînement et testé sur le reste de ces données. De manière générale ce ratio est de 80% des données pour entraîner le modèle et 20% pour le tester. Ici, nous avons décidé d'utiliser la méthode Leave-One-Out-Cross-Validation (LOOCV). Cette méthode consiste à entraîner le modèle autant de fois qu'il y a d'entité dans le set de données d'entraînement. A chaque entraînement, une donnée est retirée et utilisée comme donnée de test. Cette méthode est très utile car elle permet d'avoir plusieurs indicateurs de performance (un à chaque entraînement). La performance globale du modèle est calculée en faisant la moyenne des performances individuelles de chaque entraînement.

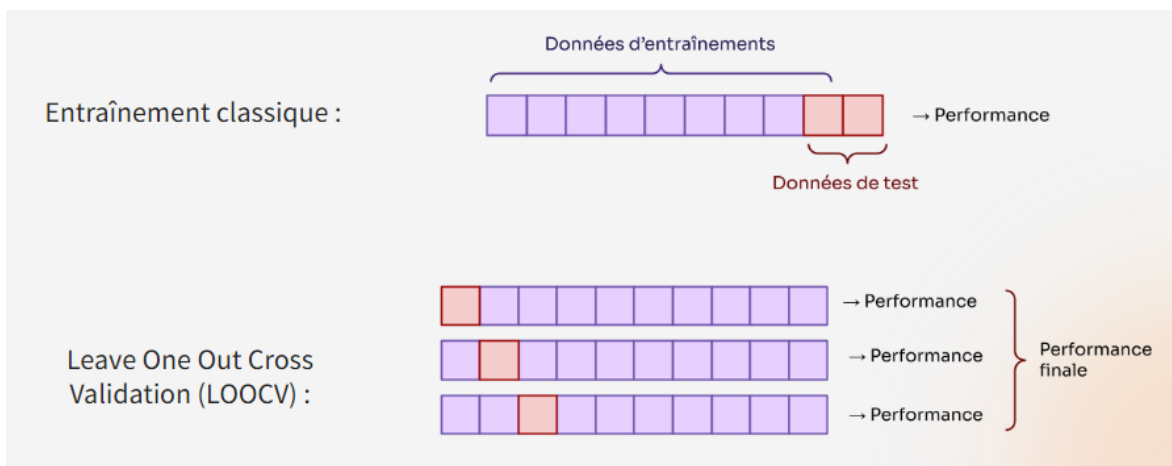


FIGURE 14 – Leave One Out Cross Validation (LOOCV)

Dans cette section nous avons expliqué les principales étapes à suivre pour répondre à la problématique de ce projet. Nous allons maintenant présenter comment nous avons mis en oeuvre ces étapes et les résultats.

4 Solution développée

4.1 Outils utilisés

Plusieurs outils informatiques ont été utilisés pour réaliser cette étude :

- * Le langage de programmation Python en version 3.9
- * Le format de données JSON pour stocker les données de questionnaires
- * Le framework Flask qui nous a permis de créer une application web afin de présenter les résultats avec les langages HTML et CSS
- * La bibliothèque python sickit-learn qui permet d'utiliser des algorithmes de machine learning [15]

Le projet se trouve dans un dépôt gitlab [16].

4.2 Strcuture du projet

Voici comment le projet est structuré :

- * Le dossier "training" contient les 24 scripts d'entraînement des différents modèles. Il contient un dossier "classifier_chains" et un dossier "OVR", chacun contenant des dossiers "detailed_values" et "mean_values".
- * Le dossier "tests_moins_un" contient 4 scripts utilisés sur les deux algorithmes multiclasse et les deux types de données (valeurs moyennes et paires de mots). Ces scripts ont servi à analyser la performance des modèles en retirant un questionnaire à chaque fois afin de voir si elle pouvait augmenter en retirant une valeur nuisible.
- * Le dossier "questionnaires" contient plusieurs fichiers JSON correspondant aux questionnaires utilisés pour l'entraînement des modèles et le test du système final. Ces questionnaires sont rangés en fonction de leur valeurs. Il y a aussi un script "questionnaires_data.py" qui permet de rentrer de nouvelles valeurs de questionnaires Attrakdiff dans les fichiers JSON.
- * Le dossier "prediction" contient le script d'entraînement du modèle le plus performant et le script "predictions_with_techniques.py" qui utilise ce modèle pour prédire de nouvelles données et les techniques d'amélioration associées aux prédictions.
- * Le dossier "app" contient l'application web qui peut être lancée via le script "app.py". Il contient aussi les templates HTML et CSS.

4.3 Traitement des données

Les données de questionnaires ont été stockées dans des fichiers JSON. Chaque questionnaire représente un élément JSON contenant les attributs "valeurs moyennes", "paire de mots", "portfolio" et "probleme". L'attribut "probleme" représente les éléments à modifier sur l'interface. Chaque élément est associé à une liste de directives à prendre en compte pour pallier le problème concerné. Par exemple pour le problème de Clarté, l'une des directives est de modifier la disposition des éléments sur le système. Ces problèmes et solutions ont été récupérés à partir des données analysées présentées à la section 3.2.1.

```
"Fixmywebsite (test2-pas de conclusions)": {
  "valeurs_moyennes": {
    "QP": 1.8,
    "QHS": 0.27,
    "QHI": 1.27,
    "ATT": 1.96
  },
  "paire_de_mots": {
    "QP": {
      "technique/humain": -1.5,
      "complique/simple": 2.5,
      "pas pratique/pratique": 2.5,
      "fastidieux/efficace": 2,
      "imprevisible/previsible": 2.5,
      "confus/clair": 3,
      "incontrolable/maitrisable": 2.5
    },
    "QHS": {
      "conventionnel/original": 1,
      "sans imagination/creatif": 1.5,
      "prudent/audacieux": 0.5,
      "conservateur/novateur": 1,
      "ennuyeux/captivant": 1,
      "peu exigeant/challenging": -2.5,
      "commun/nouveau": 1
    },
    "QHI": {
      "misole/me sociabilise": 0.5
    }
  }
}
```

FIGURE 15 – Exemple des valeurs moyennes et de paires de mots d'un questionnaire stocké dans le fichier JSON

Les conclusions des questionnaires sont stockées dans la partie "probleme". Pour ce faire, nous avons analysé les différentes conclusions mises en avant dans les évaluations Attrakdiff et les avons regroupées en catégories de problèmes plus générales. Au final, nous avons identifié 6 problèmes qui correspondent aux classes que nous utiliserons lors de la classification : clarté, créativité, intuitivité, manque d'informations, professionnalisme et esthétisme. Nous avons aussi assigné à chacun de ces problèmes les solutions mentionnées par les développeurs pour les résoudre. L'idée est que lorsque le modèle prédit un problème sur l'interface, il retourne les solutions attribuées à ce problème (voir figure 16).

```
},  
  "probleme": {  
    "Professionalisme": [  
      "Adopter des couleurs moins saturees",  
      "Adopter des couleurs plus similaires",  
      "Ajouter des microinteractions"  
    ],  
    "Esthetisme": [  
      "Choisir des couleurs plus homogenes"  
    ]  
  }  
}
```

FIGURE 16 – Exemple de problèmes et de solutions associées pour un questionnaire stocké dans le fichier JSON

L'ensemble des solutions classées par problème se trouve dans le fichier "problems_with_solutions.json". Nous avons également ajouté un fichier nommé "techniques.json" qui regroupe, par classe de problème, les outils UX que l'on peut utiliser pour améliorer l'interface. Par exemple, un problème de clarté peut être résolu en utilisant la technique du CRAP qui permet d'améliorer la mise en page d'une application web.

Comme nous avons voulu tester des modèles de machine learning en fonction des valeurs moyennes et des paires de mots, nous avons utilisé deux fichiers de données différents :

"complete_detailed_data_questionnaires.json" pour les valeurs complètes de paire de mots et

"mean_values_questionnaires.json" pour l'entraînement à partir des valeurs moyennes seulement. Nous avons décidé de ne pas mentionner le test du modèle de machine learning à partir des données de portfolio car les résultats étaient trop bas. En effet, le portfolio ne donne qu'une valeur en sortie qui est le statut et les deux moyennes hédonique et attractive. Ce nombre de caractéristique est trop peu détaillé pour permettre une bonne prédiction.

4.4 Entraînement des modèles

Chaque modèle a été entraîné dans un fichier python indépendant, nous pouvons les retrouver dans le dossier "training". A chaque fois, les données doivent être transformées pour pouvoir être utilisées dans un modèle de machine learning. Ici, les données caractéristiques (les moyennes et scores paires de mots) doivent être formattées sous forme de matrice en utilisant la bibliothèque Numpy et les classes doivent être binarisées.

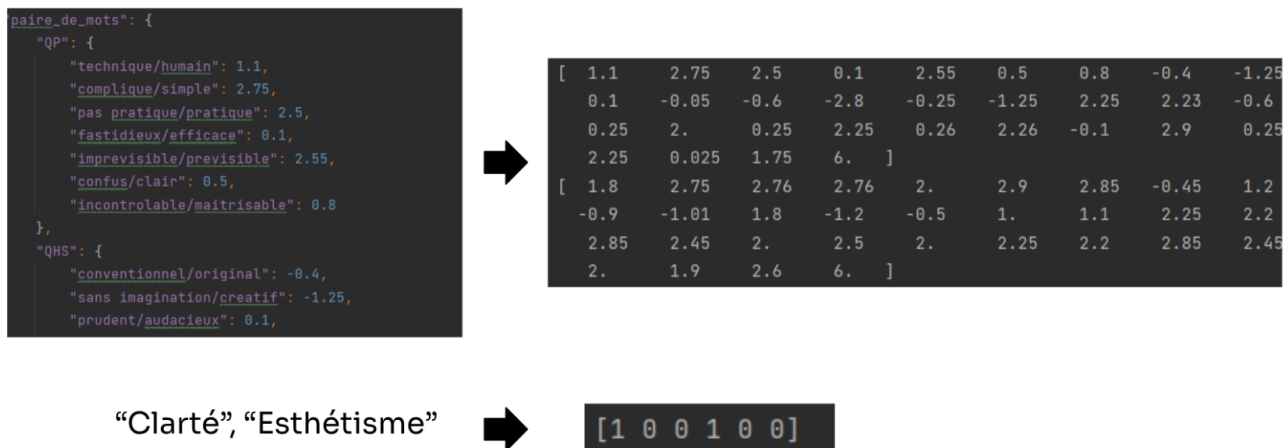


FIGURE 17 – Formattage des caractéristiques en matrice et des classes en valeurs binaires

En lançant les scripts Python, nous pouvons voir une boucle s'activer et afficher plusieurs valeurs à l'écran. Il s'agit de la boucle LOOCV qui ré-entraîne et teste le modèle à chaque fois en retirant l'un des questionnaires de l'ensemble d'entraînement. La valeur X train correspond à l'ensemble des caractéristiques des entités servant d'entraînement, X test correspond aux valeurs caractéristiques de l'entité de test, Y train correspond aux classes des entités d'entraînement et Y test correspond aux classes de l'entité de test. La valeur "prediction single" correspond à la prédiction faite par le modèle lors de l'entraînement, le but étant que "prediction single" soit égal à Y test.

```
Ytest : [[0 0 0 1 0 0 0]]
prediction single [[1. 0. 0. 0. 1. 0. 0.]]
```

FIGURE 18 – Y test correspond aux classes de la valeur de test et prediction single à la prédiction du modèle de machine learning

Le modèle peut ensuite s'entraîner à partir des données formattées. A chaque entraînement d'un modèle, deux valeurs d'exactitude sont calculées :

- * **L'exact-match ratio accuracy** : c'est une valeur qui compare chaque prédiction de modèle avec les données de test. Cette valeur augmente seulement si les valeurs prédites par le modèle sont exactement les mêmes valeurs prévues dans les données d'entraînement. Par exemple, si le modèle prédit des problèmes d'esthétisme et de clarté pour une interface alors que les réels problèmes à prédire sont l'esthétisme et le professionnalisme, le score sera de 0.
- * **Le label-based accuracy** : cette valeur prend en compte la prédiction de chaque classe et est donc moins strict. Si le modèle prédit des problèmes d'esthétisme et de clarté pour une interface alors que les réels problèmes à prédire sont l'esthétisme et le professionnalisme, le score sera de 0.5.

La méthode LOOCV est employée en faisant une boucle qui ré-entraîne le modèle à chaque fois en enlevant un questionnaire pour l'utiliser en tant que test. Les performances sont ensuite calculées en faisant les moyennes des performances individuelles.

Nous discuterons des résultats et du choix du modèle dans la section suivante.

4.5 Résultats et choix du modèle

Les figures 19 et 20 montrent les résultats des performances obtenues lors de l'entraînement des 24 modèles avec le calcul de l'exact-match ratio et du label-based accuracy respectivement.

	Réseau Bayésien	Arbres de décision	Random Forest	K-Neighbors	Régression Logistique	SVM
CHAINS - détaillés	0.0833	0	0.1667	0.0833	0	0
CHAINS - moyennes	0.0667	0	0	0.0667	0.1333	0
OVR - détaillés	0.0833	0	0.0833	0.0833	0	0
OVR - moyennes	0.0667	0	0	0.0667	0.1333	0.0667

FIGURE 19 – Performances exact match ratio accuracy de chaque modèle

Dans ce premier tableau de résultats, nous pouvons constater que le modèle le plus performant est le modèle de type Random Forest combiné à l'algorithme Classifier Chains avec les données détaillées de chaque paire de mots. Le score d'exactitude (ou accuracy) est de 0.1667.

	Réseau Bayésien	Arbres de décision	Random Forest	K-Neighbors	Régression Logistique	SVM
CHAINS - détaillés	0.6806	0.6667	0.6944	0.6667	0.5972	0.5972
CHAINS - moyennes	0.6571	0.6095	0.6857	0.7238	0.7143	0.6952
OVR - détaillés	0.6667	0.6667	0.6667	0.6667	0.5972	0.5972
OVR - moyennes	0.6857	0.6190	0.6762	0.7333	0.7143	0.7048

FIGURE 20 – Performances label-based accuracy de chaque modèle

Ce deuxième tableau nous permet de constater que le modèle K-Neighbors combiné à l'algorithme One-vs-Rest avec les données moyennes possède un meilleur taux d'exactitude en label-based avec une valeur de 0.7333. Ce score est bien plus haut ce qui est normal puisqu'il prend en compte chaque prédiction de classe individuelle au contraire du taux précédent qui augmente le score seulement si la prédiction est exacte.

Le choix du modèle s'est donc porté sur le modèle Random Forest avec l'algorithme Classifier Chains et les données détaillées, bien qu'il ne possède pas le meilleur score de label-based accuracy. En

effet en regardant la performance exacte, nous avons plus de chance de choisir un modèle plus robuste et performant. De plus, la performance de ce modèle en label-based accuracy est plutôt positive avec un score de 0.6944.

Cependant nous avons bien vu que la performance du modèle est bien trop basse pour être utilisée de manière effective (0.1667). Nous avons alors décidé d'effectuer un dernier test de performance : ré-entraîner les modèles de la même façon en éliminant l'une des données de questionnaire à chaque fois. Il peut en effet arriver que l'une des données d'entraînement fausse les résultats du modèle à cause de valeurs trop éloignées des autres. En lançant ce test, nous avons pu constater que le modèle choisi reste le plus performant, mais aussi qu'en retirant l'un des questionnaire (qui portait sur l'évaluation Attrakdiff de l'interface de Mediaserver), nous obtenions une nouvelle performance s'élevant à 0.3636 pour la prédiction exacte et 0.7576 pour la prédiction label-based. Bien que ce score n'est pas très haut, il est tout de même intéressant de noter l'importante hausse de fiabilité après le retrait de ce questionnaire.

Pour mieux comprendre cette hausse, nous avons analysé à la main les valeurs Attrakdiff de ce questionnaire. Nous avons relevé que la classe associée était le problème d'esthétisme. En comparant ce questionnaire avec d'autres qui obtenaient également le problème d'esthétisme, nous nous sommes rendus compte que les valeurs de paires de mots de ce questionnaire Mediaserver appartenant à la catégorie "Attractivité" étaient plus élevées que les scores de la même catégorie des autres questionnaires. Cette observation peut expliquer la baisse de fiabilité du modèle qui a donc capturé des valeurs d'attractivité trop différentes pour ce questionnaire par rapport aux autres questionnaires, ce qui a faussé l'algorithme d'entraînement. De plus, le fait d'avoir des bons scores pour des valeurs d'attractivité mais de conclure sur un problème d'esthétisme n'est pas logique puisque ce sont des éléments du même domaine.

Le nouveau set d'entraînement se trouve désormais dans le fichier "complete_detailed_data_without_mediaserver.json". Nous n'avons pas retiré d'autres questionnaires à cause du faible nombre de données que nous avions. En effet, lorsque l'on travaille avec un nombre trop bas de données et des modèles complexes il y a un risque d'overfitting. C'est-à-dire que le modèle ayant trop peu de données pour s'entraîner va capturer des détails de données qui ne sont pas caractéristiques. Cela peut alors mener à de mauvais résultats.

C'est finalement le modèle Random Forest combiné à Classifier Chains et aux données détaillées de paires de mots que nous avons décidé de garder pour ce projet. Le modèle a ensuite été enregistré au format pickle pour pouvoir l'invoquer et faire de la prédiction à partir de nouvelles données, sans à avoir à le ré-entraîner à chaque fois.

Nous avons ensuite développé une application web sur laquelle les utilisateurs peuvent entrer leurs résultats de questionnaire Attrakdiff pour déterminer les changements à faire sur leur interface. Cette application est présentée dans la section suivante.

4.6 Application web

Comme nous l'avons mentionné précédemment, nous avons créé une application web locale qui permet de tester le système et visualiser de manière plus esthétique les données prédites par le modèle. Cette application a été développée avec Flask, HTML et CSS et s'active en lançant le fichier "app.py". La figure 21 montre le visuel de la page d'accueil de cette application.

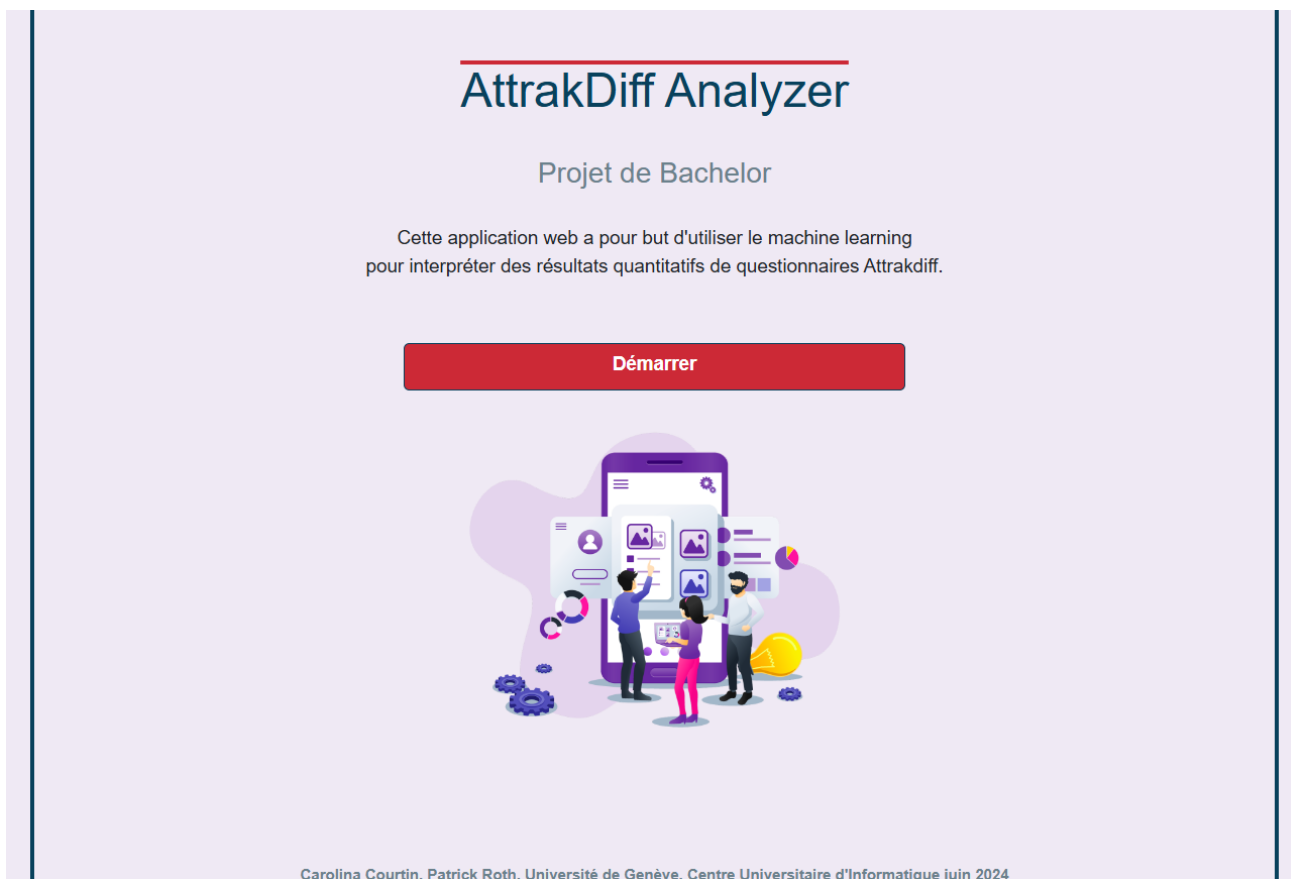


FIGURE 21 – Page d'accueil de l'application web

Quand la personne clique sur "Démarrer", elle arrive sur une nouvelle page dans laquelle elle doit remplir un formulaire avec le nom de l'interface testée et les valeurs de chaque paires de mots (voir figure 22).

AttrakDiff Analyzer

Avant de commencer la prédiction, veuillez remplir le formulaire ci-dessous avec vos résultats d'Attrakdiff

Nom de l'interface testée

Interface site Sephora

Valeurs paires de mots

Qualité pragmatique

Technique - Humain

1.2

Complicé - Simple

2

Pas pratique - Pratique

1

Fastidieux - Efficace

1

Imprévisible - Prévisible

-1

Confus - Clair

FIGURE 22 – Formulaire de données Attrakdiff

Remarque : Le formulaire demande seulement les valeurs des paires de mot puisque ce sont ces valeurs qui sont utilisées dans la prédiction, et pas les valeurs moyennes ou les valeurs du portfolio.

Une fois que le formulaire est rempli, les données sont placées dans un fichier JSON horodaté qui représente donc une interface testée avec les résultats du questionnaire Attrakdiff. Les fichiers JSON sont enregistrés dans le dossier "prediction_questionnaire_app". Ces données sont ensuite transférées au modèle de machine learning, plus précisément au script "predictions_with_techniques.json", afin de prédire les points qui posent problème dans l'interface. Ces conclusions sont ensuite affichées sur la page web avec les solutions associées ainsi que les techniques (comme le CRAP) pour résoudre ces problèmes (voir figures 23 et 24).

AttrakDiff Analyzer

Résultats de l'analyse

Problèmes identifiés :

Clarte

Professionnalisme

Solutions proposées :

- Augmenter le contraste des couleurs
- Adopter une (meilleure) nomenclature des rubriques
 - Adapter la taille des textes
- Ajouter des logos pour les boutons
- Adopter une semantique standardisee
 - Ajouter des microinteractions
- Adopter des couleurs moins saturees
- Adopter des couleurs plus similaires

FIGURE 23 – Résultats et solutions

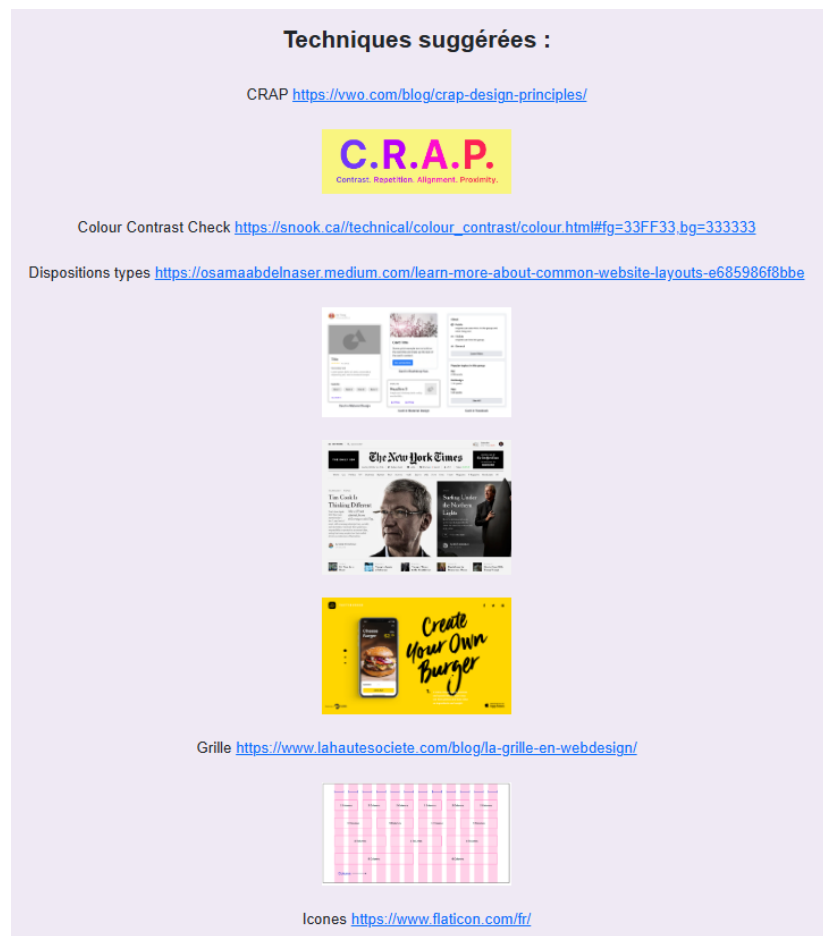


FIGURE 24 – Affichage des techniques suggérées

La figure 24 montre bien les différentes techniques que le système suggère à l'utilisateur pour améliorer son interface. On y trouve à la fois des exemples de mise en page, des conseils de cadrage, d'alignement et des articles qui expliquent plus en détails certaines techniques UX.

5 Evaluation

Pour évaluer notre système, nous avons utilisé trois évaluations d'interfaces. Le test a simplement consisté à utiliser le système avec les données de ces questionnaires et comparer le résultat de l'application avec les remarques faites par les développeurs de l'interface lors de l'évaluation réelle. Ces données d'évaluations ont été réalisées dans le cadre du cours Interface Personne Machine supervisé par le professeur Patrick Roth.

- * Evaluation du système "FixMyWebsite", des élèves ont développé une application pour partager des défauts d'un site web : le problème trouvé par le système à partir des données Attrakdiff est l'intuitivité. Ce résultat est bon car il a en effet été souligné par les développeurs, notamment avec des remarques sur la disposition de certains boutons de l'application qui n'étaient pas très visibles. Cependant, ces derniers expliquent également avoir songé à améliorer la clarté de l'interface, élément qui n'a pas été trouvé par le système.
- * Evaluation d'une application de motivation étudiante : le système a trouvé des problèmes de clarté et d'intuitivité. Ici, le résultat est bon, ce sont des problématiques qui ont bien été soulevées par les testeurs en suggérant des modifications de disposition d'éléments et de nomenclature.
- * Evaluation du site de mobilité IN de l'UNIGE : le système a trouvé des problèmes de clarté et de professionnalisme. Le raisonnement a partiellement marché puisque les testeurs de l'application ont relevé des problèmes de clarté et d'intuitivité, mais pas de professionnalisme.

Finalement, on voit que le système arrive à prédire des défauts de l'interface. Bien que les prédictions ne sont pas toujours exactes, nous pouvons quand même l'utiliser pour avoir une idée de ce qui pourrait être retravaillé. De plus, nous nous attendions à ce que les résultats ne soient pas toujours exacts car, rappelons-le, le taux d'exactitude du modèle est de 0.3636.

6 Conclusion

Le but de cette étude était de trouver un moyen d'intégrer les méthodes de machine learning au domaine de l'expérience utilisateur, et plus particulièrement utiliser un modèle de machine learning pour interpréter des résultats quantitatifs de questionnaires Attrakdiff. Après avoir défini le contexte et les modèles de machine learning, nous avons expliqué la démarche suivie qui nous a permis de sélectionner un modèle pour répondre au but du projet. Nous avons conclu sur l'utilisation d'un modèle de classification multiclasse de type Random Forest combiné à l'algorithme Classifier Chains, basé sur les données précises de chaque paires de mots Attrakdiff. Cependant nous avons également relevé le manque de données d'entraînement pour notre modèle ce qui a mené à des performances plutôt basses. C'est un aspect important à prendre en compte dans des travaux futurs pour avoir un système plus sûr. De plus, il pourrait être intéressant d'appliquer un système comme celui-ci pour évaluer d'autres types d'interfaces comme des interfaces tangibles, puisque nous nous sommes focalisés sur les interfaces de sites web. Il serait également très intéressant de s'appuyer sur d'autres types de questionnaires, notamment des questionnaires moins détaillés où une telle aide à l'interprétation serait encore plus utile.

Finalement, ce projet nous a permis de comprendre comment les développeurs d'interface pourraient être aidés pour créer des interface plus efficaces. Bien que le projet puisse être approfondi sous certains axes, il permet tout de même d'avoir une base solide de recherche dans ce domaine de machine learning appliqué à l'expérience utilisateur et plus particulièrement à l'interprétation de questionnaires d'évaluation d'interfaces.

7 Annexe

- * Dépôt GitLab où se trouve le projet <https://gitlab.unige.ch/Carolina.Courtin/projet-de-bachelor>
- * Dossier contenant les travaux de conception et d'évaluation d'interfaces du cours Interface Personne Machine (IPM) https://drive.google.com/drive/folders/13aS0oH_hAYqtqpwjyQ_f5XIuRQyFZrj?usp=sharing
- * Questionnaire Attrakdiff <https://lagrandeourse.design/blog/ux-research/methode-attrakdiff/>

Références

- [1] Effect of User Interface and User Experience on Application Sales, M A T Pratama and A T Cahyadi, <https://doi.org/10.1088/1757-899X/879/1/012133>
- [2] SUS - A quick and dirty usability scale, John Brooke, <https://www.researchgate.net/publication/228593520>*SUS_A_quick_and_dirty_usability_scale*
- [3] SUS - Construction and Evaluation of a User Experience Questionnaire, Bettina Laugwitz, Theo Held, Martin Schrepp <https://www.researchgate.net/publication/221217803>*Construction_and_Evaluation_of_a_User_Experience_Questionnaire*
- [4] A French version of the AttrakDiff scale : Translation and validation study of a user experience assessment tool C. Lallemand, V. Koenig, G. Gronier, R. Martin <https://www.sciencedirect.com/science/article/pii/S1162908815000651>
- [5] Interpreting the Results from the User Experience Questionnaire (UEQ) Using the Importance-Performance Analysis (IPA), Andreas Hinderks, Anna-Lena Meiners, Francisco José Domínguez Mayo, Jörg Thomaschewski <https://www.researchgate.net/publication/336217763>*Interpreting_the_Results_from_the_User_Experience_Questionnaire_Using_the_Importance-Performance_Analysis_IPA*
- [6] User Experience Design Using Machine Learning : A Systematic Review, Abdallah M. H. Abbas, Khairil Imran Ghauth, Choo-Yee Ting <https://ieeexplore.ieee.org/document/9770797>
- [7] Detecting Emotions Through Machine Learning for Automatic UX Evaluation, Giuseppe Desolda, Andrea Esposito, Rosa Lanzilotti, Maria Francesca Costabile <https://www.researchgate.net/publication/354119101>*Detecting_Emotions_Through_Machine_Learning_for_Automatic_UX_Evaluation*
- [8] Bayesian networks for user modeling : Predicting the user's preferences, Rebai Rim, Mohamed Amine Maalej, Adel Mahfoudhi <https://www.researchgate.net/publication/286814467>*Bayesian_networks_for_user_modeling_predicting_the_user's_preferences*
- [9] Predicting Final User Satisfaction Using Momentary UX Data and Machine Learning Techniques, Kitti Koonsanit, Nobuyuki Nishiuchi <https://www.mdpi.com/0718-1876/16/7/171>
- [10] Augmented TV : Experiencing Augmented Virtual TV in a Game-based PC Environment, Marianna Obrist, Judith Igelsböck, Florian Foerster, Joerg Hofstaetter <https://www.researchgate.net/publication/228626084>*Augmented_TV_Experiencing_Augmented_Virtual_TV_in_a_Game-based_PC_Environment*
- [11] Surgical declarative knowledge learning : concept and acceptability study A. Huauilmé, G. Dardenne, B. Labbe, M. Gelin, C. Chesneau <https://www.tandfonline.com/doi/full/10.1080/24699322.2022.2086484>
- [12] DinerRouge - Bringing Wealth and Income Inequality to the Table through a Provocative UX, Adrian Holzer, Samuel Bendahan, Bruno Kocher, Denis Gillet <https://www.researchgate.net/publication/280735965>*DinerRouge_-_Bringing_Wealth_and_Income_Inequality_to_the_Table_through_a_Provocative_UX*
- [13] Developing the Scale Adoption Framework for Evaluation (SAFE), William Green, Greg Dunn, Jettie Hoonhout <https://www.researchgate.net/publication/253457474>*Developing_the_Scale_Adoption_Framework_for_Evaluation*
- [14] FlexiGroups, Tero Jokela, Andrés Lucero <https://www.researchgate.net/publication/288492257>*FlexiGroups*
- [15] Bibliothèque Python sickit learn <https://scikit-learn.org/stable/>
- [16] Dépôt GitLab où se trouve le projet <https://gitlab.unige.ch/Carolina.Courtin/projet-de-bachelor>
- [17] Image page de garde <https://fr.freepik.com/>
- [18] Image developpement d'interfaces <https://www.winbids.fr/design-thinking/>
- [19] Image réseaux bayésiens
- [20] Image arbres de décision <https://medium.com/@shrutimisra/interpretable-ai-decision-trees-f9698e94ef9b>
- [21] Image Random Forest <https://www.geeksforgeeks.org/random-forest-classifier-using-scikit-learn/>
- [22] Image KNN <https://medium.com/@sachinsoni600517/k-nearest-neighbours-introduction-to-machine-learning-algorithms-9dbc9d9fb3b2>

- [23] Image régression logistique <https://blog.goodaudience.com/machine-learning-using-logistic-regression-in-python-with-code-ab3c7f5f3bed>
- [24] Image SVM <https://towardsdatascience.com/support-vector-machine-introduction-to-machine-learning-algorithms-934a444fca47>